

Day1 실습 실행결과

광주_3반_임해안

실습1 - 대용량 로그 스트리밍 집계

pr1.py

```
(.venv) limhaean@limhaean-ui-MacBookPro Python_data_0720_Day1 % python practice1/pr1.py

총 건수 : 200000

-- 상태코드 TOP 5 --
[('200', 159884), ('500', 8126), ('301', 8055), ('404', 8007), ('302', 8007)]
5xx: 16047건 (8.0%)
200000

=====
총 요청 수 : 200,000

5xx 오류율 : 8.0%

-- 인기 경로 TOP 5 --
/static/app.js      20,204
/health             20,152
/checkout           20,128
/cart               20,036
/products/detail    20,030

-- 시간대별 요청 수 --
00시 8,685
01시 8,543
02시 8,569
03시 8,769
04시 8,608
05시 8,515
06시 8,599
07시 8,674
08시 8,564
09시 8,729
10시 8,615
11시 8,633
12시 8,595
13시 8,607
14시 8,624
15시 8,672
16시 8,653
17시 8,613
18시 8,642
19시 7,100
20시 7,275
21시 7,203
22시 7,177
23시 7,336

-- 접속 상위 IP TOP 5 --
83.3.180.55      2
173.10.79.216    2
84.136.62.63     2
205.212.216.202  2
186.215.89.61    2

-- 한 걸음 더: 최대 메모리 비교 --
readlines : 200,000건, 26.01 MB
generator  : 200,000건, 0.05 MB
메모리 절감률: 99.8%
```

웹 로그 200,000건을 제너레이터 방식으로 집계한 결과 5xx 오류율은 8.0%로 확인되었다. 제너레이터의 최대 메모리는 0.05MB로, readlines() 방식 대비 약 99.8% 절감되었다.

실습2 - Pydantic v2 중첩 스키마 검증

pr2.py

```
• (.venv) limhaean@limhaean-ui-MacBookPro Python_data_0720_Day1 % python practice2/pr2.py
응답 상태: ok
전체 건수: 40
{
  "id": 1,
  "username": "user_001",
  "email": "user1@example.com",
  "age": 30,
  "is_active": false,
  "signup_date": "2024-07-02",
  "profile": {
    "country": "KR",
    "tier": "pro",
    "score": 92.91
  },
  "tags": []
}
0 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
1 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
2 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
3 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
4 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
5 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
6 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
7 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
8 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
9 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}

-- STEP 1: 기본 모델 검증 --
정상 데이터: id=1 username='user_001'
잘못된 데이터 감지: Input should be a valid integer, unable to parse string as an integer

-- STEP 2: 나이 범위 검증 --
정상 데이터: id=1 username='user_001' age=30
잘못된 데이터 감지: 필드=age, 사유=Input should be greater than or equal to 0

-- STEP 3: 이메일 형식 검증 --
정상 데이터: id=1 username='user_001' age=30 email='user1@example.com'
잘못된 데이터 감지: 필드=email, 사유=Value error, 올바른 이메일 형식이 아닙니다

-- STEP 4: 중첩 모델 검증 --
정상 데이터: id=1 username='user_001' age=30 email='user1@example.com' is_active=False signup_date=datetime.date(2024, 7, 2) profile=Profile(country='KR', tier='pro', score=92.91) tags=[]
중첩 모델 타입: Profile
잘못된 데이터 감지: 필드=profile.score, 사유=Input should be less than or equal to 100

-- STEP 5: 전체 데이터 검증 --
전체 40건 → 유효 36건 / 오염 4건

-- STEP 6: 오염 데이터 상세 --

```

행	ID	필드	사유
7	7	age	Input should be greater than or equal to 0
13	13	email	Value error, 올바른 이메일 형식이 아닙니다
21	21	email	Field required
29	29	profile.score	Input should be less than or equal to 100

Pydantic 모델로 사용자 데이터 40건을 검증하여 유효 36건과 오염 4건으로 분리하였다.
음수 나이, 이메일 형식 및 누락, 프로필 점수 범위 초과를 정상적으로 탐지하였다.

실습3 - asyncio 기반 비동기 수집기

pr3.py

```
● (.venv) limhaean@limhaean-ui-MacBookPro Python_data_0720_Day1 % python practice3/pr3.py
동기: 6.24초
비동기 단일 요청: {'id': 1, 'ok': True}
비동기: 60건, 0.10초
비동기 제한: 60건, 최대 동시 10건, 0.61초
타임아웃 적용: 성공 60건 / 타임아웃 0건, 0.61초
타임아웃 동작 확인: {'id': 999, 'ok': False, 'reason': 'timeout'}
요청 7 실패 (일시적 연결 오류), 1초 후 재시도
요청 13 실패 (일시적 연결 오류), 1초 후 재시도
재시도 적용: 성공 60건 / 실패 0건, 재시도 발생 2건, 1.31초
예외 격리: 성공 59건 / 실패 1건
격리된 오류: RuntimeError: 예상하지 못한 수집 오류
요청 7 실패 (일시적 연결 오류), 0.1초 후 재시도
요청 13 실패 (일시적 연결 오류), 0.1초 후 재시도
요청 55 실패 (재시도로 복구되지 않는 연결 오류), 0.1초 후 재시도
요청 55 실패 (재시도로 복구되지 않는 연결 오류), 0.2초 후 재시도
dead-letter 저장: 1건 → /Users/limhaean/workspace/skala_python/Python_data_0720_Day1/practice3/dead_letter.json
```

practice3
dead_letter.json
pr3.py

```
{ } dead_letter.json x pr2.py pr3.py
Python_data_0720_Day1 > practice3 > { } dead_letter.json > ...
1
2 {
3   "id": 55,
4   "error": "재시도로 복구되지 않는 연결 오류",
5   "attempts": 3
6 }
7
```

동기 방식은 약 6.2초가 걸렸지만 비동기 방식은 60건을 약 0.1초에 처리하였다.
Semaphore, timeout, 재시도, 예외 격리를 적용했으며 최종 실패 요청은 dead-letter
파일에 보존하였다.

- 종합실습1 - 비동기 ETL 파이프라인

pipeline.py

```
● (.venv) limhaean@imhaean-ui-MacBookPro skala_python % python Python_data_0720_Day1/total1/pipeline.py  
{'total': 60, 'valid': 60, 'invalid': 0, 'rows_saved': 60}
```

pytest -v

```
● (.venv) limhaean@imhaean-ui-MacBookPro total1 % pwd  
/Users/limhaean/workspace/skala_python/Python_data_0720_Day1/total1  
● (.venv) limhaean@imhaean-ui-MacBookPro total1 % ls  
__pycache__  models.py  output  pipeline.py  test_pipeline.py  
● (.venv) limhaean@imhaean-ui-MacBookPro total1 % pytest -v  
===== test session starts =====  
platform darwin -- Python 3.11.15, pytest-8.4.2, pluggy-1.6.0 -- /Users/limhaean/workspace/skala_python/.venv/bin/python  
cachedir: .pytest_cache  
rootdir: /Users/limhaean/workspace/skala_python/Python_data_0720_Day1/total1  
collected 6 items  
  
test_pipeline.py::test_parquet_round_trip PASSED [ 16%]  
test_pipeline.py::test_load_writes_csv_and_parquet PASSED [ 33%]  
test_pipeline.py::test_extract_returns_all_requested_ids PASSED [ 50%]  
test_pipeline.py::test_category_is_normalized PASSED [ 66%]  
test_pipeline.py::test_negative_price_is_rejected PASSED [ 83%]  
test_pipeline.py::test_valid_and_invalid_counts_match_input PASSED [100%]  
  
===== 6 passed in 0.56s =====
```

ruff check . && ruff format .

```
● (.venv) limhaean@imhaean-ui-MacBookPro total1 % ruff check . && ruff format .  
All checks passed!  
3 files left unchanged
```

output폴더에 csv, parquet 파일 생성

```
▼ total1  
  > __pycache__  
  > .pytest_cache  
  > .ruff_cache  
  ▼ output  
    products.csv  
    products.parquet  
  models.py  
  pipeline.py  
  test_pipeline.py  
● (.venv) limhaean@imhaean-ui-MacBookPro total1 % ls -lh output  
total 16  
-rw-r--r--@ 1 limhaean  staff   1.5K Jul 20 19:51 products.csv  
-rw-r--r--@ 1 limhaean  staff   3.7K Jul 20 19:51 products.parquet
```

비동기 Extract, Pydantic Transform, CSV·Parquet Load로 구성된 ETL 파이프라인을 구현하였다. 총 60건을 저장했으며 pytest 6개와 Ruff 검사·포맷을 모두 통과하였다.